



02 - 04 DECEMBER 2025

RIYADH EXHIBITION AND CONVENTION
CENTER, MALHAM, SAUDI ARABIA

Living in the namespace: Stealthy persistence in Linux

Oxmadvice – Jabr

ORGANISED BY:



IN ASSOCIATION WITH:



~# Whoami

- Jabr AlOtaibi,
- 17-years old security researcher focused on the Linux kernel, browser engines, and low level in general.
- Past-work: 5+CVEs, UxPlay, AirPlay, ChromoServers, etc.
- ...



~# Agenda

- Motivation behind this technique
- Linux namespaces 101
- Unshare'ing to persist
- Live Demo
- Conclusion



~# Motivation behind this technique

- In most Linux systems, maintaining long-term access traditionally relies on creating startup entries like systemd services, cron jobs, or adding commands to shell configuration files.
- Yes they're effective but...
- They're Noisy
- Actions get logged
- Conventional and leaves artifacts



~# Stealth is required

- For successful operations stealth is a crucial thing, and what we need is a way to stay alive on a system without leaving “obvious” traces, no extra files no reboots etc...
- Modern Linux offers built-in primitives, like namespaces that allow processes to operate in an isolated environment, while still running on the same kernel.



~# Linux namespaces 101

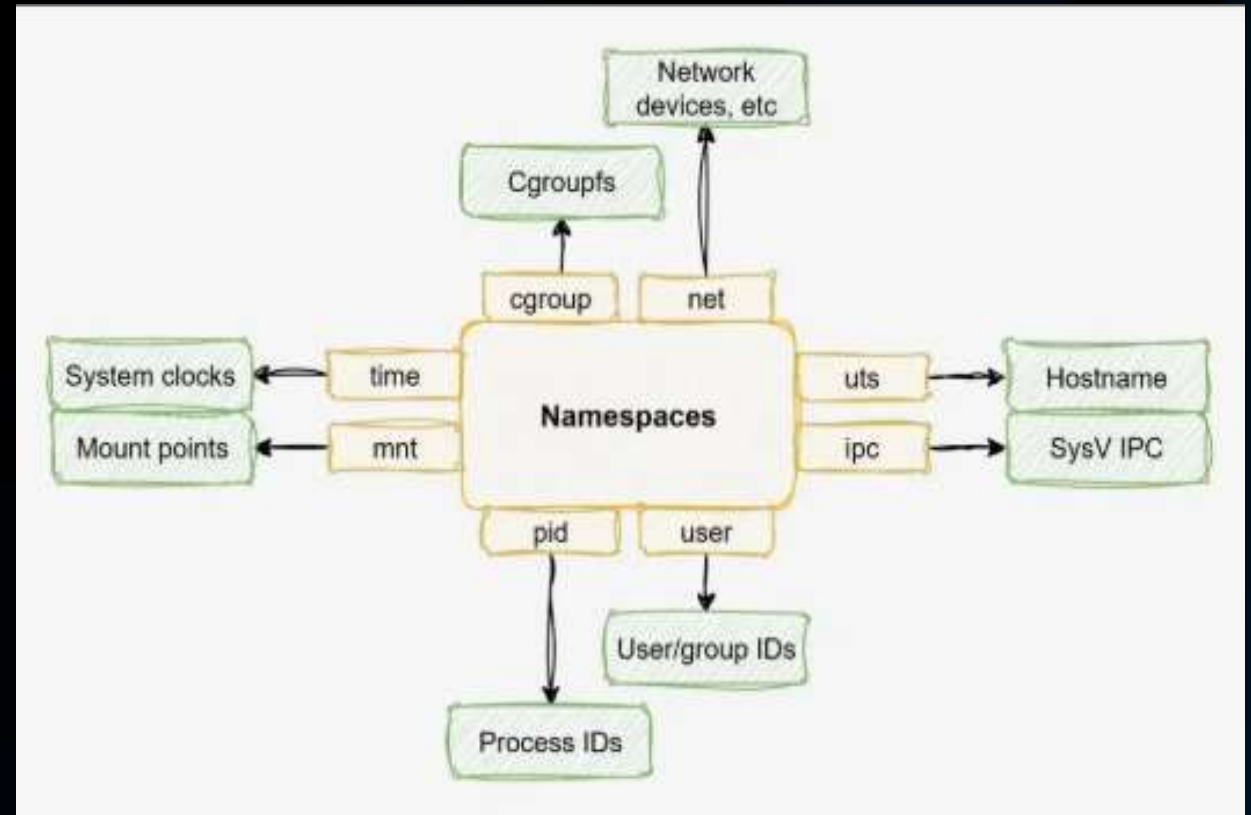


~# Understanding Linux namespaces

- To understand the technique, we first need to understand how namespaces work.
- Linux namespaces are a kernel-level feature that allow processes to see isolated views of system resources, each namespace has a type such as PID, mount points, network interfaces, IPC objects and so on.
- This isolation is the foundation of modern containers like Docker, Kubernetes and system-nspawn, as well as sandboxed envs.

~# Understanding Linux namespaces .2

- Pid = process ids
- Net = network interfaces
- Mnt = mount points and fs
- Uts = hostname/domain name
- Ipc = inter-process comms
- User = UID/GID maps
- CGROUPS = Cgroup hierarchies



~# Unshare'ing to persist



~# Unshare'ing to persist

- The core idea is to maintain a hidden presence on a Linux system without leaving obvious traces. Instead of creating services, modifying startup or even rebooting the host, we leverage existing linux namespaces to operate in an isolated environments.
- The code first scans for accessible namespaces to join in “/proc*/ns/*”

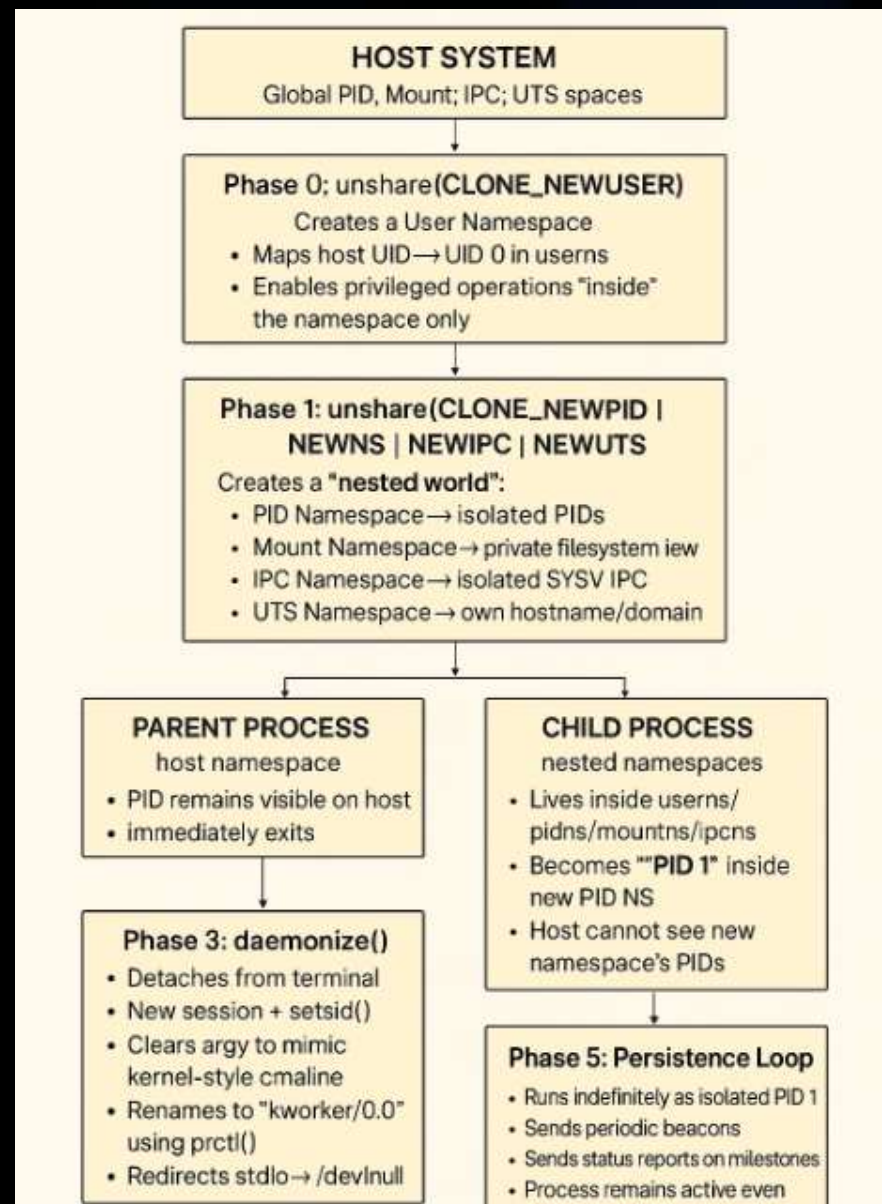
```
snuff@Bikka:~$ sudo ls /proc/*/ns/*  
/proc/773/ns/cgroup      /proc/919/ns/time_for_children  /proc/self/ns/pid  
/proc/773/ns/ipc        /proc/919/ns/user              /proc/self/ns/pid_for_children  
/proc/773/ns/mnt        /proc/919/ns/uts               /proc/self/ns/time  
/proc/773/ns/net        /proc/934/ns/cgroup            /proc/self/ns/time_for_children  
/proc/773/ns/pid        /proc/934/ns/ipc               /proc/self/ns/user  
/proc/773/ns/pid_for_children /proc/934/ns/mnt              /proc/self/ns/uts
```

~# Unshare'ing to persist .2

- And by joining these namespaces the process can “piggyback” on existing workloads and run silently alongside normal system activity, effectively hiding in plain sight.
- Once inside a namespace, the program creates its own user namespace and maps itself as root, gaining full control of the process, mount points and network interfaces within that namespace, then it detaches itself and daemonizes itself.



~# Live demo



~# Defending against this

- Defending against a fully weaponized version of this technique is difficult but not impossible.
- To defend against this technique, sysadmins can enforce strict namespace isolation and access controls, limit which users can join or create namespaces and monitor `/proc/` for unusual activities.
- And also. `sysctl -w kernel.unprivileged_usersns_clone=0`
- Which is used to control whether unprivileged users can create new user namespaces or not.



~# Finishing up



~# Finishing up

- Linux namespaces provide powerful primitives for isolation and stealth.
- This approach avoids traditional noisy persistence mechanisms like system and cron jobs, making it extremely difficult to detect and mitigate.
- Namespaces are not just for containers-they can be leveraged for advanced persistence and stealth on modern linux systems. Defenders must be aware of the risks, and monitor namespace usage and enforce strict rules.

~# Thank you for listening!

